

## 2nd REVIEW

### Student Course Enrollment and Academic Management System

|                  |            |
|------------------|------------|
| V. ABHIDEV       | 2520030325 |
| TEJ PRATAP SINGH | 2520030476 |
| G. SHASHANK      | 2520030495 |

Under the Guidance  
of

**Dr. Y Subbarayudu**

Assistant Professor  
Department of Computer Science and Engineering



**KONERU LAKSHMAIAH EDUCATION FOUNDATION**

## Implementation Overview

The portal connects student course registration with administrative academic operations. The React interface communicates with the backend through HTTP requests.

Students select a cluster, enroll in compatible sections and track their academic progress. Server checks apply the same rules to interface and direct API requests.

Administrators review student courses, publish grades and manage official CGPA records. Academic changes include a recorded reason where required.

The current implementation uses React, an Express REST API and a MySQL database. The production backend serves the built frontend from one application.

The implementation uses separate route, service and database modules.

## Software Requirements and Technology Stack

| Component   | Technology          | Role                                         |
|-------------|---------------------|----------------------------------------------|
| Frontend    | React and Vite      | Student and administrator interfaces         |
| Backend     | Node.js and Express | REST routes, validation and business rules   |
| Database    | MySQL with mysql2   | Persistent records and SQL transactions      |
| API testing | Postman             | Requests, responses and authentication tests |
| Development | VS Code and GitHub  | Source editing and version control           |

## Backend Implementation

server/app.js mounts API routes, parses JSON and handles request errors. The application exposes health and API documentation endpoints.

Route modules separate authentication, courses, enrollment, clusters and administration. Middleware checks authentication before protected handlers run.

Enrollment services apply academic rules before committing database changes. Eligibility includes semester, prerequisites, cluster and schedule checks.

mysql2 provides pooled connections and parameterized SQL queries. A transaction uses one connection for related enrollment writes.

The production Express server also serves the built React interface. Development uses Vite while production uses the built static assets.

## Database Implementation

| Record group      | Tables                             | Purpose                                |
|-------------------|------------------------------------|----------------------------------------|
| Identity          | students, admins, faculty          | Accounts, profiles and roles           |
| Course catalog    | courses, course_sections           | Course details, capacity and schedules |
| Registration      | enrollments, waitlist              | Enrollment state and seat queues       |
| Cluster selection | academic_clusters, cluster_groups  | Four clusters and section groups       |
| Academic records  | grades, attendance, cgpa_overrides | Results, attendance and official CGPA  |
| Audit history     | academic_audit                     | Record administrative academic changes |

## Authentication and Access Control

The login endpoint validates credentials and issues an eight-hour JWT session. The client includes the token in later protected API requests.

Salted bcrypt hashes protect stored passwords. Login compares the entered password with its stored hash.

Authentication middleware verifies the token before granting access. Invalid or expired tokens prevent access to protected operations.

Student ownership checks restrict access to personal academic records. A student cannot request another student's protected personal data.

Admin role checks protect grade publication, CGPA changes and audit endpoints. The backend checks permissions even if a user bypasses the interface.

## Enrollment Transaction

1. Start a transaction and lock section rows in a stable order. The stable order makes concurrent enrollment changes consistent.
2. Lock the student and check semester, cluster, prerequisites and schedule. The requested offering must belong to the student's selected cluster.
3. Check duplicate enrollment, the credit limit, capacity and waiting students. These checks decide whether the student can enroll or join a waitlist.
4. Save the enrollment and lock the student's cluster selection. Save timetable and notification changes on the same database connection.
5. Commit the transaction. Reject invalid requests without a partial enrollment. A failed operation rolls back all changes within the transaction.

## Cluster and Section Implementation

Each semester contains four clusters with 2, 3, 2 and 3 section groups. Each subject has offerings within the configured clusters.

Students choose one cluster before enrolling in a course. The semester choice links the student to a single cluster.

The planner displays course sections belonging to that selected cluster. Students may choose different groups within their selected cluster.

The backend rejects a requested section from another cluster. Direct API requests cannot bypass the cluster restriction.

Enrollment or waitlisting locks the choice. Dropping a course keeps the lock. The lock persists to prevent mixing subjects through repeated drops.

## Administrator and CGPA Operations

The student directory links each student with enrolled courses and academic details. Administrators can inspect a student's enrollment and cluster together.

Administrators publish course marks and record attendance. Published grades and attendance remain linked to student records.

Published results produce a credit-weighted GPA calculation. The number of credits determines each grade's contribution.

An official CGPA override accepts values from 0 to 10 and requires a reason. The backend validates the value before saving the override.

Clearing the override restores the calculated GPA. Academic changes enter the audit history. The directory and reporting views use the effective academic value.

## REST API Implementation

| Method | Endpoint                        | Operation                             |
|--------|---------------------------------|---------------------------------------|
| POST   | /api/auth/login                 | Authenticate student or administrator |
| GET    | /api/courses                    | Retrieve the course catalog           |
| POST   | /api/enrollments                | Enroll or request a waitlist place    |
| PUT    | /api/clusters/students/:id      | Choose a cluster                      |
| PUT    | /api/clusters/students/:id/cgpa | Set or clear official CGPA            |
| GET    | /api/admin/overview             | Read administrative statistics        |

## Postman and Database Demonstration

Import postman/Student\_Portal\_API.postman\_collection.json into Postman. The collection contains the prepared API requests.

Run Health, then Login Student. The collection saves the returned token. A successful health response confirms the API can reach the database.

Send Courses, Student Profile and List clusters and offerings. Inspect status codes and JSON responses for each request.

Compare the returned course and student records with SQL results in MySQL Workbench. An accepted change should appear in the related database record.

Use a student token on Admin Overview, then repeat after Login Admin to show role checks. This demonstrates both denied access and authorized administration.

## Validation and Testing

Core tests cover password handling, tokens and enrollment rule calculations. These checks run independently of the browser interface.

Database/API integration checks cover enrollment, waitlists and academic operations. The checks exercise persisted records and API responses.

Cluster checks verify that students cannot mix offerings from different clusters. Rejected requests should preserve existing enrollments.

Admin checks cover official CGPA changes, restored calculation and audit records. An audit record provides evidence of the administrative change.

Production build verification checks that the React application bundles successfully. Recorded results describe the tested environment rather than every deployment.

## Implementation Status and Next Steps

Implemented: authentication, enrollment, cluster restrictions and administrator controls. Student and administrator workflows connect to persistent records.

Implemented: persistent academic records, GPA calculations and audit history. The database also provides views for academic summaries and rankings.

The current release runs as one Express application backed by MySQL. This deployment keeps local startup and debugging straightforward.

Next steps include automated deployment, backups and expanded API testing. A later release needs Windows, MySQL and browser end-to-end validation.

Microservices and additional database technologies remain future scope. Architecture changes should follow measured workload requirements.

## MySQL Queries for Implementation Evidence

| SQL command                               | What it demonstrates                        |
|-------------------------------------------|---------------------------------------------|
| USE student_portal_pro;                   | Select the configured application database  |
| SHOW TABLES;                              | Inspect the relational tables               |
| SELECT * FROM courses;                    | Read persisted course offerings             |
| SELECT * FROM v_student_academic_summary; | Inspect effective academic summary values   |
| SELECT * FROM waitlist;                   | Read waiting students and section requests  |
| SHOW TRIGGERS;                            | Inspect stored database trigger definitions |

Workbench executes SQL. The Express backend uses mysql2 to query the same database.

## Source Files and Implementation Responsibilities

| File                          | Responsibility                                               |
|-------------------------------|--------------------------------------------------------------|
| server/app.js                 | Route mounting, request handling and error responses         |
| server/config/db.js           | MySQL connection pool configuration                          |
| server/services/enrollment.js | Academic eligibility, transactions and queue promotion       |
| server/db/schema.sql          | Tables, relationships, constraints and stored database logic |
| server/db/seed.js             | Insert demonstration data through a database connection      |
| README.md                     | Configuration, execution and demonstration instructions      |

## Implementation Outcomes

Student and administrator pages read records through the Express API.

The backend accepts eligible course requests and rejects invalid selections.

A semester cluster lock applies across every subject selected by the student.

Administrative CGPA changes preserve the grade-derived academic results.

Database records and audit history provide evidence of completed operations.

Further validation should cover Windows startup, MySQL and interactive browser flows.

# THANK YOU

Questions and Live Demonstration